

$$A+B=C$$

Model Selection and Optimization

$$\sqrt{a}$$

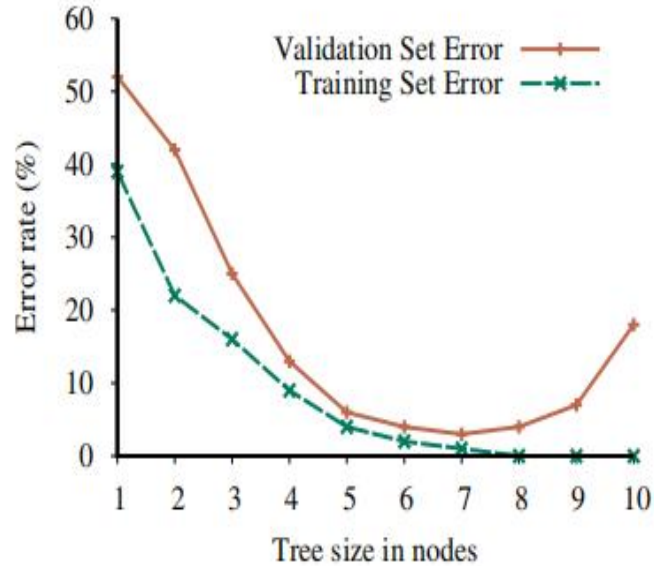
$$2^{-}$$



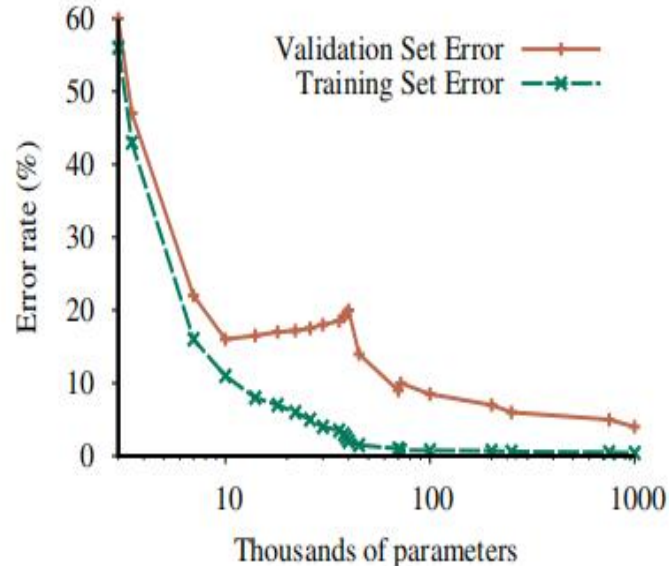
Model Selection and Optimization



Model selection



(a)



(b)

Model-Selection

picks the hyperparameter value with the lowest validation-set error.

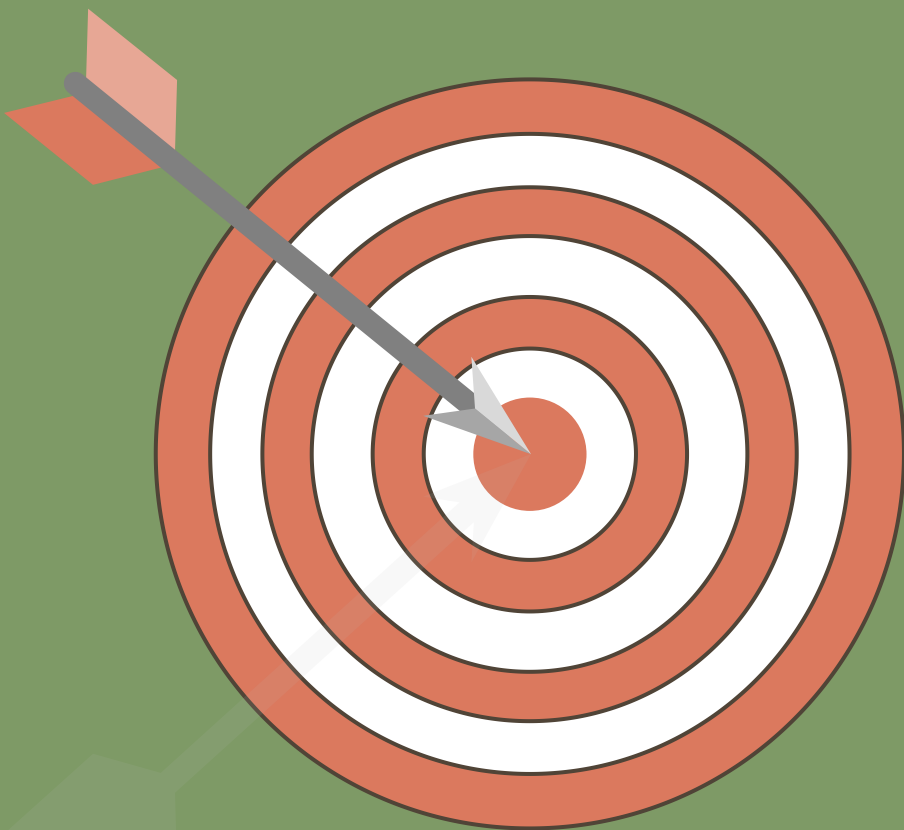
Decision Trees

hyperparameter is the number of nodes

Convolutional Neural Networks

number of regular parameters.

From error rates to loss



Absolute-value loss:

$$L1(y, \hat{y}) = |y - \hat{y}|$$

Squared-error loss:

$$L2(y, \hat{y}) = (y - \hat{y})^2$$

0/1 loss

$$(y, \hat{y}) = 0 \text{ if } y = \hat{y}, \text{ else } 1$$



Generalization Loss

Expected generalization loss for a Generalization loss hypothesis h (with respect to loss function L) is:

$$GenLoss_L(h) = \sum_{(x,y) \in \mathcal{E}} L(y, h(x)) P(x, y)$$

Best hypothesis, h^* , is the one with the minimum expected generalization loss:

$$h^* = \operatorname{argmin}_{h \in \mathcal{H}} GenLoss_L(h).$$





Empirical Loss

Uncomputable since we don't know the real-world distribution $P(x, y)$

$$EmpLoss_{L,E}(h) = \sum_{(x,y) \in E} L(y, h(x)) \frac{1}{N}.$$

The estimated best hypothesis $\hat{h}^*$ is then the one with minimum empirical loss:

$$\hat{h}^* = \operatorname{argmin}_{h \in \mathcal{H}} EmpLoss_{L,E}(h)$$





why h^* may differ from the true function, f :

1. Unrealizability (Approximation Error / Bias)

- The true function f may not be representable in the hypothesis space H .

2. Variance (Sampling Error)

- The training dataset is only a small sample of all possible data.

3. Noise

- (measurement errors, label noise, randomness in data).

4. Computational Complexity

- Even if a perfect hypothesis exists in H , finding it may be computationally intractable.





Regularization

1. An alternative approach to validation

$$\begin{aligned} \text{Cost}(h) &= \text{EmpLoss}(h) + \lambda \text{Complexity}(h) \\ \hat{h}^* &= \underset{h \in \mathcal{H}}{\operatorname{argmin}} \text{Cost}(h). \end{aligned}$$

which directly minimizes the weighted sum of empirical loss and the complexity of the hypothesis

2. process of explicitly penalizing complex hypotheses is called regularization
 - feature selection





What is a Hyperparameter?

1. Hyperparameters are “knobs we tune before training” that control the model’s learning process and complexity, and good choices are critical for generalization.

Parameters = learned from data

Hyperparameters = chosen by us (outside the training loop)





What is a Hyperparameter?

1. A parameter is something the model learns automatically during training (e.g., weights w , biases b in neural networks).
2. A hyperparameter is something set by the human designer before training begins. It controls the learning process rather than being learned itself.

Parameters = learned from data

Hyperparameters = chosen by us (outside the training loop)





Examples of Hyperparameters

1. Model complexity

- Degree of polynomial in polynomial regression
- Number of layers / neurons in a neural network
- Number of clusters k in K-means

2. Optimization-related

- Learning rate in gradient descent
- Batch size
- Number of epochs





How do we Choose Hyperparameters?

1. **Grid Search** → try all combinations (expensive).
2. **Random Search** → randomly sample hyperparameters.
3. **Cross-validation** → evaluate performance of each choice on validation sets.
4. **Bayesian optimization** → machine learning problem in itself.



$$A+B=C$$

Support vector machines

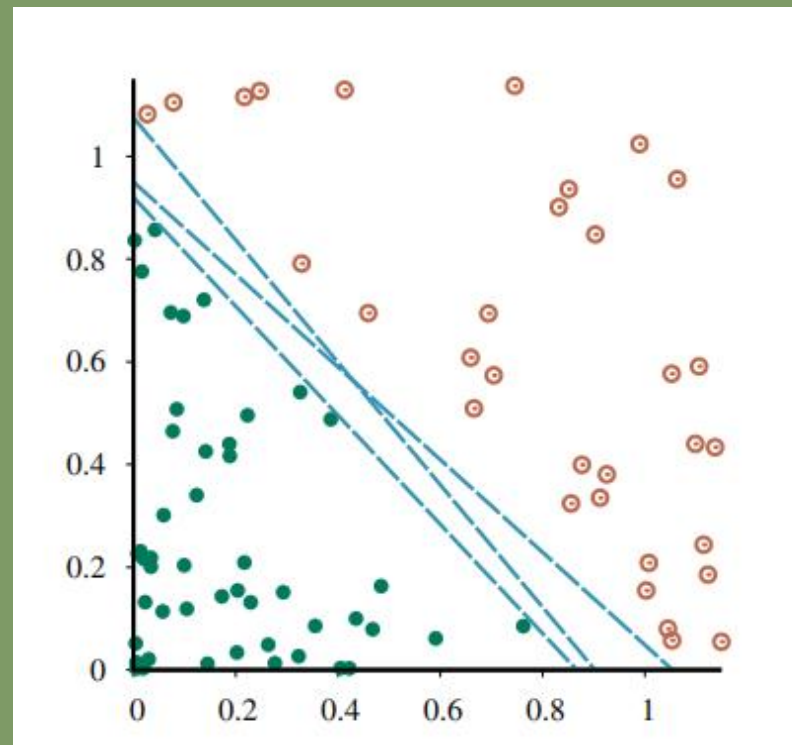
$$\sqrt{a}$$

$$2^{-}$$



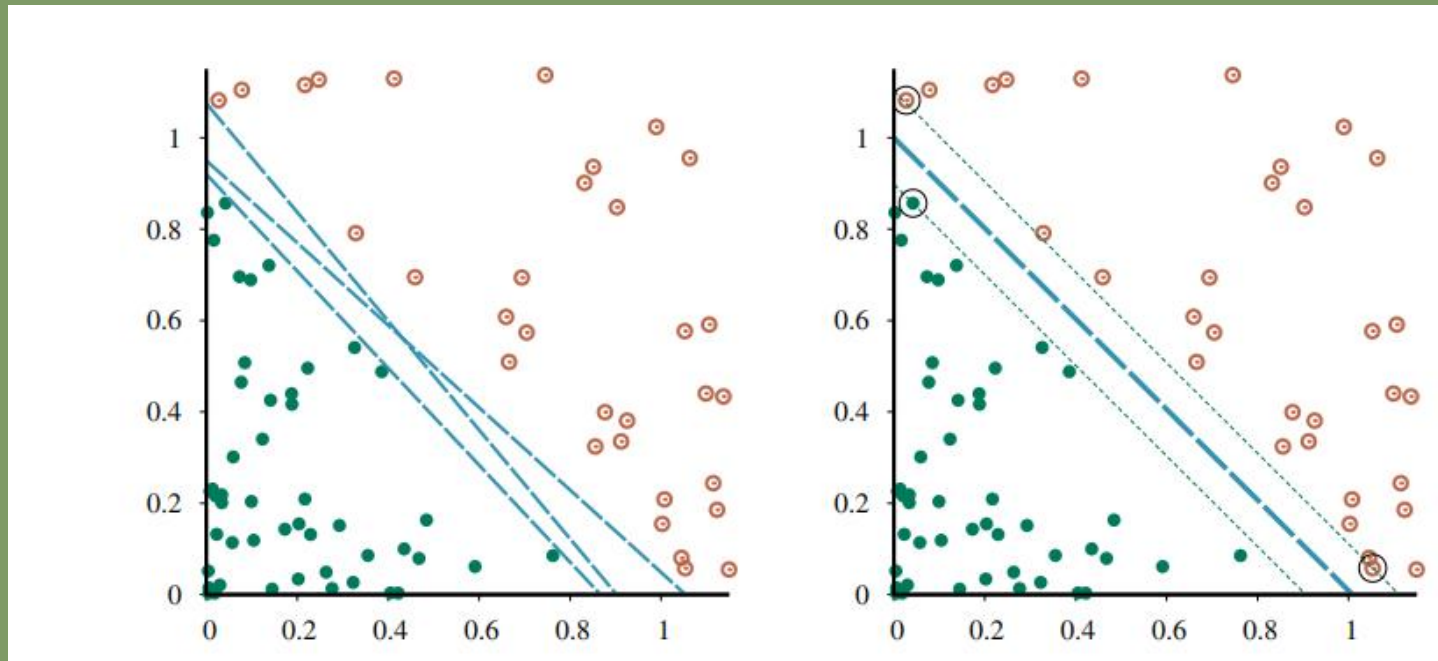


Support vector machines





Support vector machines



biggest empty cushion between the line and the closest dots





Support vector machines

- Linear separating hyperplane
- Maximum margin separator
- Nonparametric
- A higher-dimensional space- kernel trick.





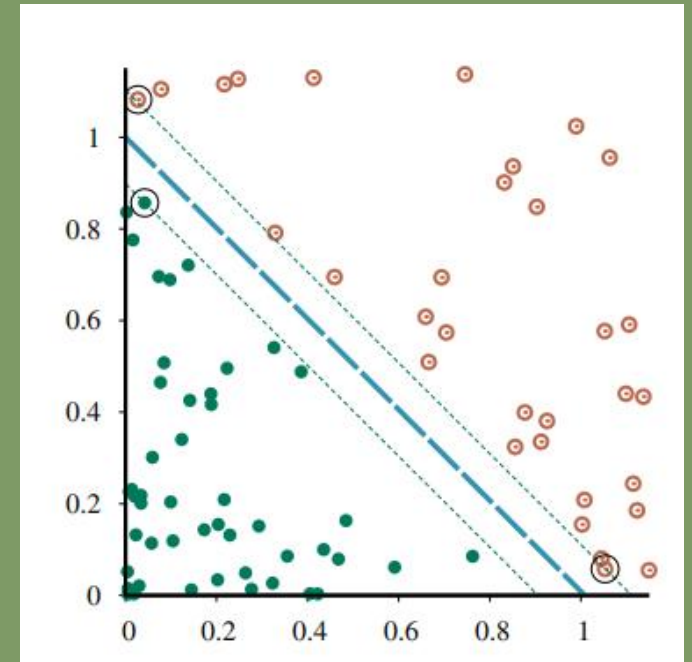
The separator and the margin

- Any straight separator can be written as $\{ x \mid w \cdot x + b = 0 \}$, $y_i \in \{+1, -1\}$.
- Distance from a point x to the line is

$$\text{dist}(x, \text{line}) = \frac{|w \cdot x + b|}{\|w\|}$$

$$y_i (w \cdot x_i + b) = 1$$

- full width between the two dashed margin lines is $2/\|w\|$





The “primal” SVM problem

- “Make the margin as wide as possible” $\Leftrightarrow$ “make $\|w\|$ as small as possible”

$$\begin{array}{ll} \min_{\mathbf{w}, b} & \frac{1}{2} \|\mathbf{w}\|^2 \\ \text{s.t.} & y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 \quad \forall i. \end{array}$$





The “dual” SVM problem

- Lagrange multipliers $\alpha_i \geq 0$

$$\max_{\alpha} \left| \sum_j \alpha_j - \frac{1}{2} \sum_{j,k} \alpha_j \alpha_k y_j y_k (x_j \cdot x_k) \right| \quad \text{s.t.} \quad \alpha_j \geq 0, \quad \sum_j \alpha_j y_j = 0.$$

- separator:

$$\mathbf{w} = \sum_j \alpha_j y_j x_j, \quad b = y_i - \sum_j \alpha_j y_j (x_j \cdot x_i) \text{ for any support vector } i.$$

- To classify a new point x :

$$f(x) = \text{sign} \left(\sum_j \alpha_j y_j (x_j \cdot x) + b \right)$$





Why the dual is nice

- dot products $x_j \cdot x_k$
- kernel $K(x_j, x_k)$

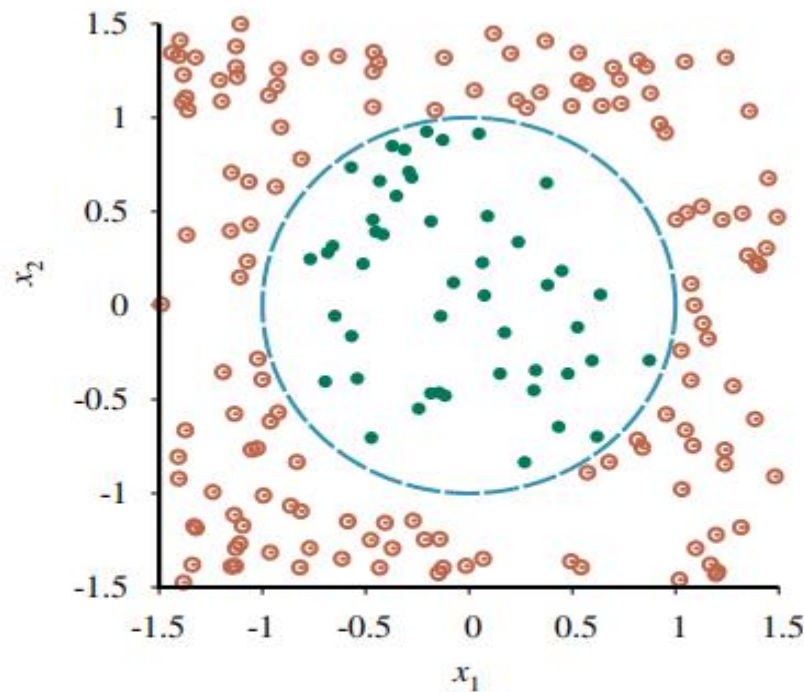


Why only support vectors matter

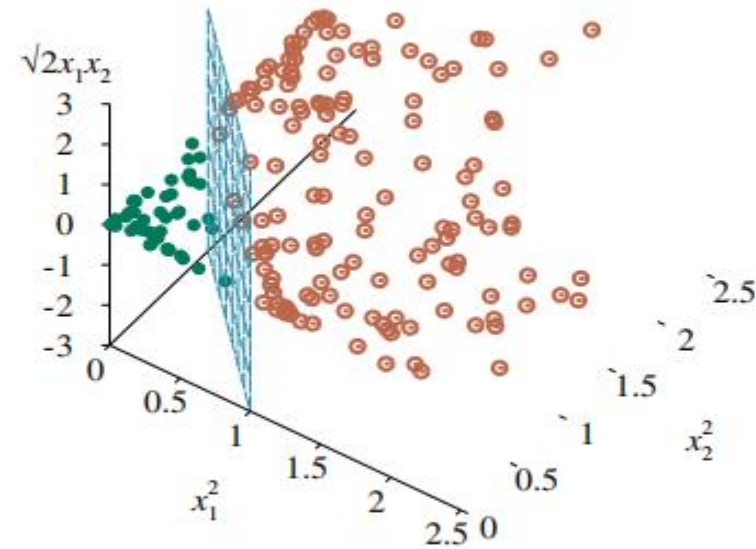
For most training points, $\alpha_j = 0$. Only points exactly on the margin boundary (the closest ones) have non-zero α_j .



Non-linearly separable case →→ feature mapping



(a)



(b)





Non-linearly separable case →→ feature mapping

- original input is $x = (x_1, x_2)$. We map it to new features:

$$f_1 = x_1^2, \quad f_2 = x_2^2, \quad f_3 = \sqrt{2} x_1 x_2$$

- Mapping data into higher dimensions
- The dot-product shortcut

$$F(x) = (f_1, f_2, f_3) = (x_1^2, x_2^2, \sqrt{2} x_1 x_2).$$

$$\begin{aligned} F(x) \cdot F(x') &= (x_1^2)(x_1'^2) + (x_2^2)(x_2'^2) + (\sqrt{2} x_1 x_2)(\sqrt{2} x_1' x_2') \\ &= x_1^2 x_1'^2 + x_2^2 x_2'^2 + 2x_1 x_2 x_1' x_2' \\ &= (x_1 x_1' + x_2 x_2')^2 \\ &= \boxed{(x \cdot x')^2}. \end{aligned}$$





Kernel function

- A kernel is a function that equals a dot product in some (possibly high dimensional) feature space, without ever computing that space explicitly. For this example,

$$K(x, x') = (x \cdot x')^2$$

acts exactly like $F(x) \cdot F(x')$. This lets algorithms like SVM build curved decision boundaries while still using efficient linear algebra.

